

Deloitte Most Asked JAVA DSA Questions

(Arrays, Strings, Linked List, Stack, Queue, Trees, BST, Graphs, DP etc)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- **75+ Interview Questions** from TCS coding rounds.
 - **Beginner to Advanced Topics:** Arrays → Strings → Linked List → Stack → Queue → Trees → BST → Graphs → DP.
 - **Code in C++ with Time & Space Complexity.**
 - **Concept Explanations** included for quick revision.
 - **Optimized Solutions** for interviews and campus placements.
-

Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on **most frequently asked questions**.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all DELOITTE rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

1. Two Sum – LeetCode 1

Description:

Find two numbers in an array that add up to a specific target. Return their indices.
Use a hash map to store complements for faster lookup.

```
class TwoSum {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int diff = target - nums[i];
            if (map.containsKey(diff))
                return new int[]{map.get(diff), i};
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}
```

□ $O(n)$ | □ $O(n)$

2. Best Time to Buy and Sell Stock – LeetCode 121

Description:

Find the maximum profit from a single buy and sell action.
Track the minimum price and compute profit differences.

```
class MaxProfit {
    public int maxProfit(int[] prices) {
        int min = prices[0], profit = 0;
        for (int p : prices) {
            min = Math.min(min, p);
            profit = Math.max(profit, p - min);
        }
        return profit;
    }
}
```

□ $O(n)$ | □ $O(1)$

3. Move Zeroes – LeetCode 283

Description:

Move all zeros to the end while maintaining the order of non-zero elements.
Use two-pointer approach for in-place swap.

```
class MoveZeroes {
    public void moveZeroes(int[] nums) {
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
```

```

        if (nums[i] != 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j++] = temp;
        }
    }
}

```

☐ O(n) | ☐ O(1)

4. Reverse String – LeetCode 344

Description:

Reverse an array of characters in place using two pointers.
Swap characters from both ends toward the center.

```

class ReverseString {
    public void reverseString(char[] s) {
        int l = 0, r = s.length - 1;
        while (l < r) {
            char temp = s[l];
            s[l++] = s[r];
            s[r--] = temp;
        }
    }
}

```

☐ O(n) | ☐ O(1)

5. Valid Palindrome – LeetCode 125

Description:

Check if a given string is a palindrome ignoring cases and non-alphanumeric characters.
Use two pointers from both ends.

```

class ValidPalindrome {
    public boolean isPalindrome(String s) {
        s = s.replaceAll("[^A-Za-z0-9]", "").toLowerCase();
        int l = 0, r = s.length() - 1;
        while (l < r)
            if (s.charAt(l++) != s.charAt(r--)) return false;
        return true;
    }
}

```

☐ O(n) | ☐ O(1)

6. Merge Two Sorted Lists – LeetCode 21

Description:

Merge two sorted linked lists into a single sorted list.

Use pointers to compare nodes one by one.

```
class MergeLists {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0), curr = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val < l2.val) { curr.next = l1; l1 = l1.next; }
            else { curr.next = l2; l2 = l2.next; }
            curr = curr.next;
        }
        curr.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

□ $O(m+n)$ | □ $O(1)$

7. Maximum Depth of Binary Tree – LeetCode 104

Description:

Find the height of a binary tree recursively using DFS.

Return $1 + \max(\text{left depth}, \text{right depth})$.

```
class MaxDepth {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

□ $O(n)$ | □ $O(h)$

8. Fibonacci Number – LeetCode 509

Description:

Compute the Nth Fibonacci number using recursion or dynamic programming.

Avoid recomputation using memoization or iteration.

```
class Fibonacci {
    public int fib(int n) {
        if (n <= 1) return n;
        int a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            int c = a + b;
            a = b; b = c;
        }
    }
}
```

```

        return b;
    }
}

```

□ $O(n)$ | □ $O(1)$

9. Contains Duplicate – LeetCode 217

Description:

Check if the array contains any duplicate element using a HashSet.

If element repeats, return true.

```

class ContainsDuplicate {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int n : nums)
            if (!set.add(n)) return true;
        return false;
    }
}

```

□ $O(n)$ | □ $O(n)$

10. Intersection of Two Arrays – LeetCode 349

Description:

Return intersection of two arrays as a unique set of elements.

Use HashSets to store and check presence.

```

class Intersection {
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> set1 = new HashSet<>(), set2 = new HashSet<>();
        for (int n : nums1) set1.add(n);
        for (int n : nums2) if (set1.contains(n)) set2.add(n);
        return set2.stream().mapToInt(Integer::intValue).toArray();
    }
}

```

□ $O(n+m)$ | □ $O(n)$

11. Single Number – LeetCode 136

Description:

Find the element that appears only once where every other element appears twice.

Use XOR property.

```

class SingleNumber {
    public int singleNumber(int[] nums) {

```

```

        int res = 0;
        for (int n : nums) res ^= n;
        return res;
    }
}

```

□ $O(n)$ | □ $O(1)$

12. Plus One – LeetCode 66

Description:

Increment a number represented as an array of digits.
Handle carry propagation from least significant digit.

```

class PlusOne {
    public int[] plusOne(int[] digits) {
        for (int i = digits.length - 1; i >= 0; i--) {
            if (digits[i] < 9) { digits[i]++; return digits; }
            digits[i] = 0;
        }
        int[] res = new int[digits.length + 1];
        res[0] = 1;
        return res;
    }
}

```

□ $O(n)$ | □ $O(1)$

13. Maximum Subarray – LeetCode 53

Description:

Find contiguous subarray with largest sum using Kadane's algorithm.
Keep track of running and max sum.

```

class MaxSubarray {
    public int maxSubArray(int[] nums) {
        int maxSoFar = nums[0], curr = nums[0];
        for (int i = 1; i < nums.length; i++) {
            curr = Math.max(nums[i], curr + nums[i]);
            maxSoFar = Math.max(maxSoFar, curr);
        }
        return maxSoFar;
    }
}

```

□ $O(n)$ | □ $O(1)$

14. Remove Duplicates from Sorted Array – LeetCode 26

Description:

Remove duplicates from a sorted array in place and return new length.
Use slow-fast pointer approach.

```
class RemoveDuplicates {
    public int removeDuplicates(int[] nums) {
        int j = 1;
        for (int i = 1; i < nums.length; i++)
            if (nums[i] != nums[i-1])
                nums[j++] = nums[i];
        return j;
    }
}
```

□ $O(n)$ | □ $O(1)$

15. Climbing Stairs – LeetCode 70

Description:

Count ways to reach the top of stairs taking 1 or 2 steps.
Use Fibonacci-like DP approach.

```
class ClimbStairs {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int a = 1, b = 2;
        for (int i = 3; i <= n; i++) {
            int c = a + b;
            a = b; b = c;
        }
        return b;
    }
}
```

□ $O(n)$ | □ $O(1)$

16. Palindrome Number – LeetCode 9

Description:

Check if a number reads the same backward and forward without converting to string.
Reverse half of the number.

```
class PalindromeNumber {
    public boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
```

```

        x /= 10;
    }
    return x == rev || x == rev / 10;
}
}

```

□ $O(\log n)$ | □ $O(1)$

17. Valid Parentheses – LeetCode 20

Description:

Check if parentheses are balanced using a stack.

Push open brackets, pop when matching close bracket found.

```

class ValidParentheses {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') stack.push(c);
            else if (stack.isEmpty() ||
                (c == ')' && stack.pop() != '(') ||
                (c == '}' && stack.pop() != '{') ||
                (c == ']' && stack.pop() != '['))
                return false;
        }
        return stack.isEmpty();
    }
}

```

□ $O(n)$ | □ $O(n)$

18. Merge Sorted Array – LeetCode 88

Description:

Merge two sorted arrays into the first array in place.

Start from end to avoid overwriting.

```

class MergeSortedArray {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m-1, j = n-1, k = m+n-1;
        while (j >= 0)
            nums1[k--] = (i >= 0 && nums1[i] > nums2[j]) ? nums1[i--] :
nums2[j--];
    }
}

```

□ $O(m+n)$ | □ $O(1)$

19. Search Insert Position – LeetCode 35

Description:

Find index where target should be inserted in sorted array using binary search.

```
class SearchInsert {
    public int searchInsert(int[] nums, int target) {
        int l = 0, r = nums.length - 1;
        while (l <= r) {
            int mid = l + (r - l) / 2;
            if (nums[mid] == target) return mid;
            if (nums[mid] < target) l = mid + 1;
            else r = mid - 1;
        }
        return l;
    }
}
```

□ $O(\log n)$ | □ $O(1)$

20. Implement strStr() – LeetCode 28

Description:

Return the index of first occurrence of substring in a string.

Use sliding window check.

```
class StrStr {
    public int strStr(String haystack, String needle) {
        if (needle.isEmpty()) return 0;
        for (int i = 0; i <= haystack.length() - needle.length(); i++)
            if (haystack.substring(i, i + needle.length()).equals(needle))
                return i;
        return -1;
    }
}
```

□ $O((n-m+1) * m)$ | □ $O(1)$

21. Remove Element – LeetCode 27

Description:

Remove all occurrences of a given value in-place.

Return new length.

```
class RemoveElement {
    public int removeElement(int[] nums, int val) {
        int j = 0;
        for (int n : nums)
            if (n != val)
                nums[j++] = n;
        return j;
    }
}
```

```
    }  
}
```

☐ O(n) | ☐ O(1)

22. Missing Number – LeetCode 268

Description:

Find the missing number in range [0, n].

Use XOR of indices and numbers.

```
class MissingNumber {  
    public int missingNumber(int[] nums) {  
        int xor = nums.length;  
        for (int i = 0; i < nums.length; i++)  
            xor ^= i ^ nums[i];  
        return xor;  
    }  
}
```

☐ O(n) | ☐ O(1)

23. Power of Two – LeetCode 231

Description:

Check if a number is power of two using bit manipulation.

Only one bit should be set in binary form.

```
class PowerOfTwo {  
    public boolean isPowerOfTwo(int n) {  
        return n > 0 && (n & (n - 1)) == 0;  
    }  
}
```

☐ O(1) | ☐ O(1)

24. Reverse Linked List – LeetCode 206

Description:

Reverse a singly linked list iteratively.

Rewire pointers to previous node.

```
class ReverseList {  
    public ListNode reverseList(ListNode head) {  
        ListNode prev = null;  
        while (head != null) {  
            ListNode next = head.next;  
            head.next = prev;  
            prev = head;  
            head = next;  
        }  
        return prev;  
    }  
}
```

```

        prev = head;
        head = next;
    }
    return prev;
}
}

```

□ O(n) | □ O(1)

25. Maximum Product of Two Elements – LeetCode 1464

Description:

Find the max product (nums[i]-1)*(nums[j]-1).

Sort array or find top two max values.

```

class MaxProduct {
    public int maxProduct(int[] nums) {
        int first = 0, second = 0;
        for (int n : nums) {
            if (n > first) { second = first; first = n; }
            else if (n > second) second = n;
        }
        return (first - 1) * (second - 1);
    }
}

```

□ O(n) | □ O(1)

26) Maximum Subarray — LeetCode 53

Description: Find the contiguous subarray with the largest sum (can be negative). Use Kadane's dynamic greedy update of current and global max.

```

class MaxSubarray {
    public int maxSubArray(int[] nums) {
        int max = nums[0], cur = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);
            max = Math.max(max, cur);
        }
        return max;
    }
}

```

Time: O(n) | Space: O(1)

27) Merge Sorted Array — LeetCode 88

Description: Merge `nums2` into `nums1` in-place (`nums1` has extra space). Fill from end to avoid overwriting.

```
class MergeSortedArray {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (j >= 0) {
            if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
            else nums1[k--] = nums2[j--];
        }
    }
}
```

Time: $O(m+n)$ | Space: $O(1)$

28) Search in Rotated Sorted Array — LeetCode 33

Description: Find target in array rotated at unknown pivot using modified binary search: detect sorted half each step.

```
class SearchRotatedArray {
    public int search(int[] nums, int target) {
        int l=0, r=nums.length-1;
        while (l<=r) {
            int mid = l + (r-l)/2;
            if (nums[mid]==target) return mid;
            if (nums[l] <= nums[mid]) {
                if (target >= nums[l] && target < nums[mid]) r = mid-1;
                else l = mid+1;
            } else {
                if (target > nums[mid] && target <= nums[r]) l = mid+1;
                else r = mid-1;
            }
        }
        return -1;
    }
}
```

Time: $O(\log n)$ | Space: $O(1)$

29) Search a 2D Matrix — LeetCode 74

Description: Matrix with rows/cols sorted; treat as flattened array and binary-search across virtual indices.

```
class Search2DMatrix {
    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix.length==0) return false;
        int m=matrix.length, n=matrix[0].length;
```

```

        int l=0, r=m*n-1;
        while (l<=r) {
            int mid=(l+r)/2;
            int val = matrix[mid/n][mid%n];
            if (val==target) return true;
            if (val < target) l = mid+1;
            else r = mid-1;
        }
        return false;
    }
}

```

Time: $O(\log(mn))$ | Space: $O(1)$

30) Find Peak Element — LeetCode 162

Description: Element greater than neighbors; use binary search by comparing mid with mid+1 to go uphill.

```

class FindPeak {
    public int findPeakElement(int[] nums) {
        int l=0, r=nums.length-1;
        while (l<r) {
            int mid = (l+r)/2;
            if (nums[mid] < nums[mid+1]) l = mid+1;
            else r = mid;
        }
        return l;
    }
}

```

Time: $O(\log n)$ | Space: $O(1)$

31) Rotate Image — LeetCode 48

Description: Rotate $n \times n$ matrix 90° clockwise in-place: transpose then reverse each row.

```

class RotateImage {
    public void rotate(int[][] matrix) {
        int n=matrix.length;
        for (int i=0;i<n;i++)
            for (int j=i;j<n;j++) {
                int t = matrix[i][j]; matrix[i][j]=matrix[j][i];
                matrix[j][i]=t;
            }
        for (int i=0;i<n;i++){
            for (int j=0;j<n/2;j++){
                int t = matrix[i][j]; matrix[i][j]=matrix[i][n-1-j];
                matrix[i][n-1-j]=t;
            }
        }
    }
}

```

Time: $O(n^2)$ | Space: $O(1)$

32) Set Matrix Zeroes — LeetCode 73

Description: If an element is 0, set entire row and column to 0. Use first row/col as markers to achieve $O(1)$ extra space.

```
class SetMatrixZeroes {
    public void setZeroes(int[][] matrix) {
        int m=matrix.length, n=matrix[0].length;
        boolean firstCol=false;
        for (int i=0;i<m;i++) {
            if (matrix[i][0]==0) firstCol=true;
            for (int j=1;j<n;j++) if (matrix[i][j]==0) {
                matrix[i][0]=0; matrix[0][j]=0;
            }
        }
        for (int i=m-1;i>=0;i--) {
            for (int j=n-1;j>=1;j--) if (matrix[i][0]==0 ||
matrix[0][j]==0) matrix[i][j]=0;
            if (firstCol) matrix[i][0]=0;
        }
    }
}
```

Time: $O(mn)$ | Space: $O(1)$

33) Spiral Matrix — LeetCode 54

Description: Traverse matrix in spiral order, shrinking boundaries (top, bottom, left, right) each loop.

```
import java.util.*;
class SpiralMatrix {
    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> res = new ArrayList<>();
        if (matrix.length==0) return res;
        int top=0, bottom=matrix.length-1, left=0, right=matrix[0].length-1;
        while (top<=bottom && left<=right) {
            for (int j=left;j<=right;j++) res.add(matrix[top][j]);
            top++;
            for (int i=top;i<=bottom;i++) res.add(matrix[i][right]);
            right--;
            if (top<=bottom) for (int j=right;j>=left;j--)
res.add(matrix[bottom][j]);
            bottom--;
            if (left<=right) for (int i=bottom;i>=top;i--)
res.add(matrix[i][left]);
            left++;
        }
        return res;
    }
}
```

```
}
```

Time: $O(mn)$ | Space: $O(1)$ extra (output not counted)

34) Rotate Array — LeetCode 189

Description: Rotate array to the right by k steps; reverse whole array, then reverse first k and last $n-k$ segments.

```
class RotateArray {
    public void rotate(int[] nums, int k) {
        int n=nums.length; k%=n;
        reverse(nums,0,n-1); reverse(nums,0,k-1); reverse(nums,k,n-1);
    }
    private void reverse(int[] a,int l,int r){ while(l<r){int
t=a[l];a[l]=a[r];a[r]=t;l++;r--;} }
}
```

Time: $O(n)$ | Space: $O(1)$

35) Valid Sudoku — LeetCode 36

Description: Check rows, columns, and 3×3 boxes for duplicates using boolean markers or sets.

```
class ValidSudoku {
    public boolean isValidSudoku(char[][] board) {
        boolean[][] row = new boolean[9][9], col = new boolean[9][9], box =
new boolean[9][9];
        for (int i=0;i<9;i++) for (int j=0;j<9;j++) {
            if (board[i][j]=='.') continue;
            int num = board[i][j]-'1', b = (i/3)*3 + j/3;
            if (row[i][num] || col[j][num] || box[b][num]) return false;
            row[i][num]=col[j][num]=box[b][num]=true;
        }
        return true;
    }
}
```

Time: $O(1)$ (81 cells) | Space: $O(1)$

36) Word Search — LeetCode 79

Description: Determine whether a word exists in grid moving horizontally/vertically without reusing a cell — backtracking/DFS.

```
class WordSearch {
    public boolean exist(char[][] board, String word) {
```

```

        int m=board.length, n=board[0].length;
        for (int i=0;i<m;i++) for (int j=0;j<n;j++) if (dfs(board, word, 0,
i, j)) return true;
        return false;
    }
    private boolean dfs(char[][] b, String w, int idx, int i, int j) {
        if (idx==w.length()) return true;
        if (i<0||j<0||i>=b.length||j>=b[0].length||b[i][j]!=w.charAt(idx))
return false;
        char t=b[i][j]; b[i][j]='#';
        boolean found = dfs(b,w,idx+1,i+1,j)||dfs(b,w,idx+1,i-
1,j)||dfs(b,w,idx+1,i,j+1)||dfs(b,w,idx+1,i,j-1);
        b[i][j]=t; return found;
    }
}

```

Time: $O(mn * 4^L)$ worst (L =word length) | Space: $O(L)$ recursion

37) Permutation (all permutations) — LeetCode 46

Description: Generate all permutations of distinct numbers via backtracking swapping or used[] flags.

```

import java.util.*;
class Permutations {
    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        back(nums,0,res);
        return res;
    }
    private void back(int[] a,int idx,List<List<Integer>> res){
        if (idx==a.length) {
            List<Integer> cur=new ArrayList<>();
            for(int v:a) cur.add(v);
            res.add(cur);
            return;
        }
        for (int i=idx;i<a.length;i++){
            swap(a,i,idx); back(a,idx+1,res); swap(a,i,idx);
        }
    }
    private void swap(int[] a,int i,int j){int t=a[i];a[i]=a[j];a[j]=t;}
}

```

Time: $O(n*n!)$ | Space: $O(n)$ recursion + output

38) Permutations II (with duplicates) — LeetCode 47

Description: Generate unique permutations for array with duplicates: sort then backtrack skipping used duplicate branches.

```

import java.util.*;
class Permutations2 {

```

```

    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res=new ArrayList<>();
        boolean[] used=new boolean[nums.length];
        back(nums,new ArrayList<>(),used,res);
        return res;
    }
    private void back(int[] a,List<Integer> cur,boolean[]
used,List<List<Integer>> res){
        if (cur.size()==a.length){ res.add(new ArrayList<>(cur)); return; }
        for (int i=0;i<a.length;i++){
            if (used[i]) continue;
            if (i>0 && a[i]==a[i-1] && !used[i-1]) continue;
            used[i]=true; cur.add(a[i]);
            back(a,cur,used,res);
            used[i]=false; cur.remove(cur.size()-1);
        }
    }
}

```

Time: $O(n \cdot n!)$ | Space: $O(n)$ recursion + output

39) Generate Parentheses — LeetCode 22

Description: Produce all well-formed parentheses strings of n pairs — backtracking with counts of open/close used.

```

import java.util.*;
class GenerateParentheses {
    public List<String> generateParenthesis(int n) {
        List<String> res=new ArrayList<>();
        back(new StringBuilder(),0,0,n,res);
        return res;
    }
    private void back(StringBuilder sb,int open,int close,int
n,List<String> res){
        if (sb.length()==2*n){ res.add(sb.toString()); return;}
        if (open<n){ sb.append('('); back(sb,open+1,close,n,res);
sb.deleteCharAt(sb.length()-1); }
        if (close<open){ sb.append(')'); back(sb,open,close+1,n,res);
sb.deleteCharAt(sb.length()-1); }
    }
}

```

Time: Catalan number approx $O(4^n / \sqrt{n})$ output bound | Space: $O(n)$

40) Combinations (n choose k) — LeetCode 77

Description: Generate all k -sized combinations out of $1..n$ via backtracking with start index.

```

import java.util.*;
class Combinations {
    public List<List<Integer>> combine(int n, int k) {

```

```

        List<List<Integer>> res=new ArrayList<>();
        back(1,n,k,new ArrayList<>(),res);
        return res;
    }
    private void back(int start,int n,int k,List<Integer>
cur,List<List<Integer>> res){
        if (k==0){ res.add(new ArrayList<>(cur)); return; }
        for (int i=start;i<=n;i++){
            cur.add(i); back(i+1,n,k-1,cur,res); cur.remove(cur.size()-1);
        }
    }
}

```

Time: $O(C(n,k)*k)$ | Space: $O(k)$ recursion + output

41) Linked List Cycle II (find start) — LeetCode 142

Description: Detect cycle start using Floyd's algorithm and meeting-reset trick: after meeting, move one to head and advance both to find entry.

```

class LinkedListCycle2 {
    public ListNode detectCycle(ListNode head) {
        ListNode slow=head, fast=head;
        while (fast!=null && fast.next!=null) { slow=slow.next;
fast=fast.next.next; if (slow==fast) break; }
        if (fast==null || fast.next==null) return null;
        slow=head;
        while (slow!=fast){ slow=slow.next; fast=fast.next; }
        return slow;
    }
}

```

Time: $O(n)$ | Space: $O(1)$

42) Reverse Nodes in k-Group — LeetCode 25

Description: Reverse nodes of a linked list k at a time; if fewer than k remain, leave as is. Use pointer jumps and reverse sublists.

```

class ReverseKGroup {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode dummy=new ListNode(0); dummy.next=head;
        ListNode prev=dummy, end=dummy;
        while (true) {
            for (int i=0;i<k && end!=null;i++) end=end.next;
            if (end==null) break;
            ListNode start=prev.next, next=end.next;
            end.next=null;
            prev.next = reverse(start);
            start.next = next;
            prev = start;
            end = prev;
        }
    }
}

```

```

        return dummy.next;
    }
    private ListNode reverse(ListNode head){
        ListNode prev=null, curr=head;
        while(curr!=null){ ListNode n=curr.next; curr.next=prev; prev=curr;
curr=n; }
        return prev;
    }
}

```

Time: $O(n)$ | Space: $O(1)$

43) Add Two Numbers — LeetCode 2

Description: Sum two numbers represented as reverse-order linked lists; produce sum list with carry.

```

class AddTwoNumbers {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy=new ListNode(0), p=dummy;
        int carry=0;
        while (l1!=null || l2!=null || carry!=0) {
            int s = carry + (l1!=null?l1.val:0) + (l2!=null?l2.val:0);
            p.next = new ListNode(s%10);
            carry = s/10;
            p = p.next;
            if (l1!=null) l1=l1.next;
            if (l2!=0) {} // no-op to satisfy environment
            if (l2!=null) l2=l2.next;
        }
        return dummy.next;
    }
}

```

Time: $O(\max(m,n))$ | Space: $O(\max(m,n))$ output

44) Remove Nth Node From End — LeetCode 19

Description: Use two-pointer gap of n to delete nth from end in one pass with dummy head for edge cases.

```

class RemoveNthFromEnd {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy=new ListNode(0); dummy.next=head;
        ListNode fast=dummy, slow=dummy;
        for (int i=0;i<=n;i++) fast=fast.next;
        while (fast!=null) { fast=fast.next; slow=slow.next; }
        slow.next = slow.next.next;
        return dummy.next;
    }
}

```

Time: $O(n)$ | Space: $O(1)$

45) Swap Nodes in Pairs — LeetCode 24

Description: Swap every two adjacent nodes in linked list (iteratively with dummy or recursively).

```
class SwapPairs {
    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(0); dummy.next = head;
        ListNode prev = dummy;
        while (prev.next != null && prev.next.next != null) {
            ListNode a = prev.next, b = a.next;
            prev.next = b; a.next = b.next; b.next = a;
            prev = a;
        }
        return dummy.next;
    }
}
```

Time: $O(n)$ | Space: $O(1)$

46) Intersection of Two Linked Lists — LeetCode 160

Description: Find node where two singly linked lists intersect by two-pointer switching trick; if no intersection, both become null.

```
class Intersection {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        if (headA == null || headB == null) return null;
        ListNode a = headA, b = headB;
        while (a != b) { a = (a == null) ? headB : a.next; b =
        (b == null) ? headA : b.next; }
        return a;
    }
}
```

Time: $O(m+n)$ | Space: $O(1)$

47) Implement Queue using Stacks — LeetCode 232

Description: Implement queue operations via two stacks: inStack for enqueue, outStack for dequeue amortized $O(1)$.

```
import java.util.*;
class MyQueue {
    private Stack<Integer> in = new Stack<>(), out = new Stack<>();
    public void push(int x) { in.push(x); }
    public int pop() { peek(); return out.pop(); }
```

```

    public int peek(){ if(out.isEmpty()){ while(!in.isEmpty())
out.push(in.pop()); } return out.peek(); }
    public boolean empty(){ return in.isEmpty() && out.isEmpty(); }
}

```

Time: Amortized $O(1)$ | Space: $O(n)$

48) Implement Stack using Queues — LeetCode 225

Description: Implement stack (LIFO) using queues: push by enqueueing then rotating queue so new element is front.

```

import java.util.*;
class MyStack {
    private Queue<Integer> q = new ArrayDeque<>();
    public void push(int x){ q.offer(x); int sz=q.size(); while(sz-->1)
q.offer(q.poll()); }
    public int pop(){ return q.poll(); }
    public int top(){ return q.peek(); }
    public boolean empty(){ return q.isEmpty(); }
}

```

Time: push $O(n)$, pop/top $O(1)$ | Space: $O(n)$

49) Min Stack — LeetCode 155

Description: Stack that returns minimum in $O(1)$; maintain pair-of-values or an auxiliary stack tracking mins.

```

import java.util.*;
class MinStack {
    private Stack<Integer> s = new Stack<>(), mins = new Stack<>();
    public void push(int x){ s.push(x); if (mins.isEmpty() || x<=mins.peek())
mins.push(x); }
    public void pop(){ int v=s.pop(); if (v==mins.peek()) mins.pop(); }
    public int top(){ return s.peek(); }
    public int getMin(){ return mins.peek(); }
}

```

Time: $O(1)$ each | Space: $O(n)$

50) Climbing Stairs — LeetCode 70

Description: Ways to climb n stairs with 1 or 2 steps: Fibonacci DP iterative constant space.

```

class ClimbStairs {
    public int climbStairs(int n) {
        if (n<=1) return 1;
        int a=1,b=1;

```

```

        for (int i=2;i<=n;i++){ int c=a+b; a=b; b=c; }
        return b;
    }
}

```

Time: $O(n)$ | Space: $O(1)$

51) House Robber — LeetCode 198

Description: Max loot with no adjacent houses: DP with two variables representing include/exclude.

```

class HouseRobber {
    public int rob(int[] nums) {
        int incl=0, excl=0;
        for (int n:nums) {
            int new_excl = Math.max(excl, incl);
            incl = excl + n;
            excl = new_excl;
        }
        return Math.max(incl, excl);
    }
}

```

Time: $O(n)$ | Space: $O(1)$

52) House Robber II — LeetCode 213

Description: Circular houses; either exclude first or exclude last — compute two linear robber runs and take max.

```

class HouseRobber2 {
    public int rob(int[] nums) {
        if (nums.length==1) return nums[0];
        return Math.max(robRange(nums,0,nums.length-2),
robRange(nums,1,nums.length-1));
    }
    private int robRange(int[] a,int l,int r){
        int incl=0, excl=0;
        for (int i=l;i<=r;i++){
            int new_excl=Math.max(excl,incl);
            incl = excl + a[i];
            excl = new_excl;
        }
        return Math.max(incl,excl);
    }
}

```

Time: $O(n)$ | Space: $O(1)$

53) Pow(x, n) — LeetCode 50

Description: Compute x^n using fast exponentiation, handle negative n by using reciprocal.

```
class Pow {
    public double myPow(double x, int n) {
        long N = n;
        if (N<0){ x=1/x; N=-N; }
        double ans=1;
        while (N>0) {
            if ((N&1)==1) ans *= x;
            x *= x; N >>= 1;
        }
        return ans;
    }
}
```

Time: $O(\log n)$ | Space: $O(1)$

54) Kth Largest Element in an Array — LeetCode 215

Description: Use min-heap of size k or Quickselect to find kth largest (heap solution below).

```
import java.util.*;
class KthLargest {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int n:nums) {
            pq.offer(n);
            if (pq.size()>k) pq.poll();
        }
        return pq.peek();
    }
}
```

Time: $O(n \log k)$ | Space: $O(k)$

55) K Closest Points to Origin — LeetCode 973

Description: Return k points with smallest Euclidean distance; use max-heap of size k or Quickselect on distances.

```
import java.util.*;
class KClosest {
    public int[][] kClosest(int[][] points, int k) {
        PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->Integer.compare(dist(b),dist(a)));
        for (int[] p: points) {
            pq.offer(p);
            if (pq.size()>k) pq.poll();
        }
        int[][] res = new int[k][2];
    }
}
```

```

        for (int i=k-1;i>=0;i--) res[i]=pq.poll();
        return res;
    }
    private int dist(int[] p){ return p[0]*p[0]+p[1]*p[1]; }
}

```

Time: $O(n \log k)$ | Space: $O(k)$

56) Top K Frequent Elements — LeetCode 347

Description: Return k elements with highest frequency; use bucket sort ($O(n)$) or heap.

```

import java.util.*;
class TopKFrequent {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer,Integer> m = new HashMap<>();
        for (int n:nums) m.put(n,m.getOrDefault(n,0)+1);
        List<Integer>[] buckets = new List[nums.length+1];
        for (int key: m.keySet()){
            int f=m.get(key);
            if (buckets[f]==null) buckets[f]=new ArrayList<>();
            buckets[f].add(key);
        }
        int[] res=new int[k]; int idx=0;
        for (int i=buckets.length-1;i>=0 && idx<k;i--) if
(buckets[i]!=null) for (int v: buckets[i]) { res[idx++]=v; if (idx==k)
break; }
        return res;
    }
}

```

Time: $O(n)$ | Space: $O(n)$

57) Topological Sort / Course Schedule (II) — LeetCode 210

Description: Produce ordering of courses given prerequisites using Kahn's BFS topological sort; detect cycles if ordering incomplete.

```

import java.util.*;
class CourseSchedule2 {
    public int[] findOrder(int numCourses, int[][] pre) {
        List<List<Integer>> g = new ArrayList<>();
        for (int i=0;i<numCourses;i++) g.add(new ArrayList<>());
        int[] indeg = new int[numCourses];
        for (int[] p:pre){ g.get(p[1]).add(p[0]); indeg[p[0]]++; }
        Queue<Integer> q = new ArrayDeque<>();
        for (int i=0;i<numCourses;i++) if (indeg[i]==0) q.add(i);
        int[] res = new int[numCourses]; int idx=0;
        while (!q.isEmpty()) {
            int u=q.poll(); res[idx++]=u;
            for (int v:g.get(u)){ if (--indeg[v]==0) q.add(v); }
        }
        return idx==numCourses?res:new int[0];
    }
}

```

```
    }  
}
```

Time: $O(V+E)$ | Space: $O(V+E)$

58) Merge k Sorted Lists — LeetCode 23

Description: Merge k sorted linked lists using a min-heap (priority queue) on current heads or divide-and-conquer.

```
import java.util.*;  
class MergeKLists {  
    public ListNode mergeKLists(ListNode[] lists) {  
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b)->a.val-  
b.val);  
        for (ListNode l:lists) if (l!=null) pq.offer(l);  
        ListNode dummy=new ListNode(0), tail=dummy;  
        while (!pq.isEmpty()) {  
            ListNode n=pq.poll(); tail.next=n; tail=n;  
            if (n.next!=null) pq.offer(n.next);  
        }  
        return dummy.next;  
    }  
}
```

Time: $O(N \log k)$ (N =total nodes) | Space: $O(k)$

59) Binary Search Tree Iterator — LeetCode 173

Description: Iterator that returns next smallest element in BST using stack storing left spine for amortized $O(1)$ next.

```
import java.util.*;  
class BSTIterator {  
    private Deque<TreeNode> st = new ArrayDeque<>();  
    public BSTIterator(TreeNode root){ pushLeft(root); }  
    private void pushLeft(TreeNode n){ while(n!=null){ st.push(n);  
n=n.left; } }  
    public int next(){ TreeNode t=st.pop(); if(t.right!=null)  
pushLeft(t.right); return t.val; }  
    public boolean hasNext(){ return !st.isEmpty(); }  
}
```

Time: next amortized $O(1)$ | Space: $O(h)$

60) Convert Sorted Array to BST — LeetCode 108

Description: Build height-balanced BST from sorted array by recursive mid selection.

```

class SortedArrayToBST {
    public TreeNode sortedArrayToBST(int[] nums) {
        return build(nums, 0, nums.length-1);
    }
    private TreeNode build(int[] a, int l, int r) {
        if (l > r) return null;
        int m = (l+r)/2;
        TreeNode node = new TreeNode(a[m]);
        node.left = build(a, l, m-1);
        node.right = build(a, m+1, r);
        return node;
    }
}

```

Time: $O(n)$ | Space: $O(\log n)$ recursion

61) Binary Tree Zigzag Level Order — LeetCode 103

Description: Level order traversal alternating direction per level (use deque or reverse each level).

```

import java.util.*;
class ZigzagLevelOrder {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Queue<TreeNode> q = new ArrayDeque<>(); q.offer(root);
        boolean leftToRight = true;
        while (!q.isEmpty()) {
            int sz = q.size();
            LinkedList<Integer> level = new LinkedList<>();
            for (int i = 0; i < sz; i++) {
                TreeNode t = q.poll();
                if (leftToRight) level.addLast(t.val); else
                    level.addFirst(t.val);
                if (t.left != null) q.add(t.left);
                if (t.right != null) q.add(t.right);
            }
            res.add(level); leftToRight = !leftToRight;
        }
        return res;
    }
}

```

Time: $O(n)$ | Space: $O(n)$

62) Serialize and Deserialize Binary Tree — LeetCode 297

Description: Convert binary tree to string and back (use BFS or preorder with null markers).
Below: preorder with commas.

```

import java.util.*;
class Codec {
    public String serialize(TreeNode root) {

```

```

        StringBuilder sb=new StringBuilder();
        build(root,sb); return sb.toString();
    }
    private void build(TreeNode r,StringBuilder sb){
        if (r==null){ sb.append("null,"); return; }
        sb.append(r.val).append(",");
        build(r.left,sb); build(r.right,sb);
    }
    public TreeNode deserialize(String data) {
        Queue<String> q=new ArrayDeque<>(Arrays.asList(data.split(",")));
        return build(q);
    }
    private TreeNode build(Queue<String> q){
        String s=q.poll();
        if (s.equals("null")) return null;
        TreeNode node=new TreeNode(Integer.parseInt(s));
        node.left=build(q); node.right=build(q);
        return node;
    }
}

```

Time: $O(n)$ | Space: $O(n)$

63) Path Sum — LeetCode 112

Description: Determine if there's a root-to-leaf path with given sum — DFS subtracting node values.

```

class PathSum {
    public boolean hasPathSum(TreeNode root, int sum) {
        if (root==null) return false;
        if (root.left==null && root.right==null) return root.val==sum;
        return hasPathSum(root.left,sum-root.val) ||
        hasPathSum(root.right,sum-root.val);
    }
}

```

Time: $O(n)$ | Space: $O(h)$ recursion

64) Path Sum II — LeetCode 113

Description: Return all root-to-leaf paths with given sum — DFS with path list backtracking.

```

import java.util.*;
class PathSum2 {
    public List<List<Integer>> pathSum(TreeNode root, int sum) {
        List<List<Integer>> res=new ArrayList<>();
        dfs(root,sum,new ArrayList<>(),res);
        return res;
    }
    private void dfs(TreeNode r,int s,List<Integer> cur,List<List<Integer>>
res){
        if (r==null) return;

```

```

        cur.add(r.val);
        if (r.left==null && r.right==null && r.val==s) res.add(new
ArrayList<>(cur));
        dfs(r.left,s-r.val,cur,res); dfs(r.right,s-r.val,cur,res);
        cur.remove(cur.size()-1);
    }
}

```

Time: $O(n)$ | Space: $O(h + \text{output})$

65) Construct Binary Tree from Preorder and Inorder — LeetCode 105

Description: Rebuild tree using preorder root order + inorder left/right partition; use hashmap for inorder indexes.

```

import java.util.*;
class BuildTree {
    private Map<Integer,Integer> idx = new HashMap<>();
    private int preIdx;
    public TreeNode buildTree(int[] preorder, int[] inorder) {
        preIdx=0; for (int i=0;i<inorder.length;i++) idx.put(inorder[i],i);
        return helper(preorder,0,inorder.length-1);
    }
    private TreeNode helper(int[] pre,int l,int r){
        if (l>r) return null;
        int val = pre[preIdx++]; TreeNode root = new TreeNode(val);
        int k = idx.get(val);
        root.left = helper(pre,l,k-1);
        root.right = helper(pre,k+1,r);
        return root;
    }
}

```

Time: $O(n)$ | Space: $O(n)$

66) Construct Binary Tree from Inorder and Postorder — LeetCode 106

Description: Similar to above but take root from postorder end and split using inorder hashmap.

```

import java.util.*;
class BuildTree2 {
    private Map<Integer,Integer> idx = new HashMap<>();
    private int postIdx;
    public TreeNode buildTree(int[] inorder,int[] postorder){
        postIdx = postorder.length-1; for(int i=0;i<inorder.length;i++)
idx.put(inorder[i],i);
        return helper(inorder, postorder, 0, inorder.length-1);
    }
    private TreeNode helper(int[] in,int[] post,int l,int r){
        if (l>r) return null;
        int val = post[postIdx--]; TreeNode root = new TreeNode(val);
        int k = idx.get(val);

```

```

        root.right = helper(in, post, k+1, r);
        root.left = helper(in, post, l, k-1);
        return root;
    }
}

```

Time: $O(n)$ | Space: $O(n)$

67) Lowest Common Ancestor of BST — LeetCode 235

Description: For BST, compare node values: if both $<$ root go left, both $>$ root go right, else root is LCA.

```

class LCA_BST {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p,
TreeNode q) {
        while (root!=null) {
            if (p.val < root.val && q.val < root.val) root=root.left;
            else if (p.val>root.val && q.val>root.val) root=root.right;
            else return root;
        }
        return null;
    }
}

```

Time: $O(h)$ | Space: $O(1)$

68) Lowest Common Ancestor of Binary Tree — LeetCode 236

Description: Generic binary tree LCA using recursion returning non-null from subtrees to identify split.

```

class LCA_BT {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p,
TreeNode q) {
        if (root==null || root==p || root==q) return root;
        TreeNode l = lowestCommonAncestor(root.left,p,q);
        TreeNode r = lowestCommonAncestor(root.right,p,q);
        if (l!=null && r!=null) return root;
        return l!=null?l:r;
    }
}

```

Time: $O(n)$ | Space: $O(h)$

69) Binary Tree Maximum Path Sum — LeetCode 124

Description: Path can start & end anywhere; compute for each node max path through node and update global max using postorder.

```

class MaxPathSum {
    private int max = Integer.MIN_VALUE;
    public int maxPathSum(TreeNode root) { dfs(root); return max; }
    private int dfs(TreeNode r){
        if (r==null) return 0;
        int left = Math.max(0, dfs(r.left));
        int right = Math.max(0, dfs(r.right));
        max = Math.max(max, left + right + r.val);
        return Math.max(left, right) + r.val;
    }
}

```

Time: $O(n)$ | Space: $O(h)$

70) Word Ladder (Length) — LeetCode 127

Description: Shortest transformation length from beginWord to endWord changing one char at a time; BFS on word graph using generic states.

```

import java.util.*;
class WordLadder {
    public int ladderLength(String beginWord, String endWord, List<String>
wordList) {
        Set<String> dict = new HashSet<>(wordList);
        if (!dict.contains(endWord)) return 0;
        Queue<String> q = new ArrayDeque<>(); q.add(beginWord);
        int level=1;
        while(!q.isEmpty()){
            int sz=q.size();
            for (int i=0;i<sz;i++){
                String w=q.poll();
                if (w.equals(endWord)) return level;
                char[] arr = w.toCharArray();
                for (int j=0;j<arr.length;j++){
                    char old=arr[j];
                    for (char c='a'; c<='z'; c++){
                        arr[j]=c; String ns=new String(arr);
                        if (dict.remove(ns)) q.add(ns);
                    }
                    arr[j]=old;
                }
            }
            level++;
        }
        return 0;
    }
}

```

Time: $O(M * N * 26)$ approx | Space: $O(N)$

71) Minimum Window Substring — LeetCode 76

Description: Find smallest substring of s containing all chars of t using sliding window & freq counts, expand and shrink window.

```
import java.util.*;
class MinWindow {
    public String minWindow(String s,String t) {
        if (s.length()==0||t.length()==0) return "";
        int[] need = new int[128];
        for (char c:t.toCharArray()) need[c]++;
        int required = 0; for (int v:need) if (v>0) required++;
        int l=0,r=0,formed=0; int[] window=new int[128];
        int[] ans={-1,0,0};
        while (r<s.length()){
            char c=s.charAt(r); window[c]++;
            if (window[c]==need[c]) formed++;
            while (l<=r && formed==required){
                if (ans[0]==-1 || r-l+1 < ans[0]) { ans[0]=r-l+1; ans[1]=l;
                ans[2]=r; }
                char ch=s.charAt(l); window[ch]--;
                if (window[ch]<need[ch]) formed--;
                l++;
            }
            r++;
        }
        return ans[0]==-1?"":s.substring(ans[1], ans[2]+1);
    }
}
```

Time: $O(|s| + |t|)$ | Space: $O(1)$ (fixed alphabet)

72) Evaluate Reverse Polish Notation — LeetCode 150

Description: Evaluate RPN expression using stack of integers and applying operators when encountered.

```
import java.util.*;
class EvalRPN {
    public int evalRPN(String[] tokens) {
        Stack<Integer> st=new Stack<>();
        for (String tok:tokens) {
            if ("+-*/".contains(tok)) {
                int b=st.pop(), a=st.pop();
                int res=tok.equals("+")?a+b:tok.equals("-")?a-
b:tok.equals("*")?a*b:a/b;
                st.push(res);
            } else st.push(Integer.parseInt(tok));
        }
        return st.pop();
    }
}
```

Time: $O(n)$ | Space: $O(n)$

73) Largest Rectangle in Histogram — LeetCode 84

Description: Use monotonic increasing stack of indices to compute max area where heights are extended between smaller bars.

```
import java.util.*;
class LargestRectangle {
    public int largestRectangleArea(int[] h) {
        Stack<Integer> st=new Stack<>();
        int n=h.length, max=0;
        for (int i=0;i<=n;i++){
            int cur = (i==n)?0:h[i];
            while(!st.isEmpty() && cur < h[st.peek()]) {
                int height=h[st.pop()];
                int width = st.isEmpty() ? i : i - st.peek() - 1;
                max = Math.max(max, height * width);
            }
            st.push(i);
        }
        return max;
    }
}
```

Time: $O(n)$ | Space: $O(n)$

74) Trapping Rain Water — LeetCode 42

Description: Compute water trapped between bars; two-pointer approach tracks max left/right and accumulates $\min(\text{maxL}, \text{maxR}) - \text{height}$.

```
class TrappingRain {
    public int trap(int[] h) {
        int l=0,r=h.length-1, maxL=0,maxR=0, ans=0;
        while (l<r) {
            if (h[l] < h[r]) {
                if (h[l] >= maxL) maxL=h[l]; else ans += maxL - h[l];
                l++;
            } else {
                if (h[r] >= maxR) maxR=h[r]; else ans += maxR - h[r];
                r--;
            }
        }
        return ans;
    }
}
```

Time: $O(n)$ | Space: $O(1)$

75) Maximum Product Subarray — LeetCode 152

Description: Track current max and min (because negative flips sign) at each step and update global max.

```
class MaxProduct {
    public int maxProduct(int[] nums) {
        int curMax=nums[0], curMin=nums[0], res=nums[0];
        for (int i=1;i<nums.length;i++){
            int x=nums[i];
            if (x<0) { int t=curMax; curMax=curMin; curMin=t; }
            curMax = Math.max(x, curMax * x);
            curMin = Math.min(x, curMin * x);
            res = Math.max(res, curMax);
        }
        return res;
    }
}
```

Time: $O(n)$ | Space: $O(1)$

Congratulations!

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more **coding tutorials, challenges, tips, and tech content** on Instagram: **@SYNTAXERROR**. Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.